ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 4

N15. Հեռախոսների խանութ

Աշխատանքի նպատակը

- Սովորել օգտագործել string տիպի դաշտեր, `struct` կառուցվածք և կառուցվածքների զանգվածներ C++ լեզվում .
- Ստեղծել առանձին ֆունկցիաներ զանգվածի մշակման, որոնման և հաշվարկների համար

Աշխատանքի պահանջը

- Ստեղծել `Phone` կառուցվածք՝ հետևյալ դաշտերով՝ `brand`, `model`, `operatingSystem` և `price` .
- Մուտքագրել 10 հեռախոսի տվյալներ].
- Ֆունկցիաների միջոցով իրականացնել .
    1. Արտածել բոլոր հեռախոսները
    2. Գտնել ամեն էժան հեռախոսը
    3. Արտածել տրված օպերացիոն համակարգով հեռախոսները ,
    4. Հաշվել տրված գնային սահմանից ցածր հեռախոսների քանակը

Ընդհանուր պահանջներ

1. Յուրաքանչյուր խնդրում սահմանել առանձին `struct` տիպ և օգտագործել դաշտերի իմաստալից անգլերեն անուններ].
2. Մուտքագրումը, հաշվարկը, որոնումը կամ փոփոխումը կազմակերպել առնվազն երկու առանձին ֆունկցիայով.
3. Միայն ընթերցող ֆունկցիաներում կիրառել `const` հղում կամ `const` զանգված: Փոփոխող ֆունկցիաներում կիրառել հղում կամ ցուցիչ .
4. Ստուգել քանակների, գների, գնահատականների և այլ թվային տվյալների թույլատրելի սահմանները
5. Դասակարգելիս փոխանակել ամբողջ կառուցվածքային տարր.
6. Արտածումը ձևավորել ընթեռնելի վերնագրերով: Կոդը լրացնել կարճ մեկնաբանություններով

Տեսական մաս

Կառուցվածքային տիպերը (`struct`) թույլ են տալիս միավորել տարբեր տիպերի տվյալներ (օրինակ՝ `string` և `double`) մեկ ընդհանուր օբյեկտի մեջ: Կառուցվածքների զանգվածների հետ աշխատելիս յուրաքանչյուր հեռախոսի տվյալները մշակվում են առանձին
ֆունկցիաների միջոցով՝ ապահովելով կոդի մոդուլային և կառուցվածքային լինելը;

```
#include <iostream>
```

```
#include <string>
#include <vector>
#include <iomanip>
using namespace std;

// Phone կառուցվածք
struct Phone {
    string brand;           // Արտադրող (Բրենդ)
    string model;           // Մոդել
    string operatingSystem; // Օպերացիոն համակարգ (OS)
    double price;           // Գին (դրամ)
};

// 1. Արտածել բոլոր հեռախոսները
void printAllPhones(const vector<Phone>& phones) {
    cout << "\n============================================================\n";
    cout << "              ԲՈԼՈՐ ՀԵՌԱԽՈՍՆԵՐԻ ՑԱՆԿԸ                  \n";
    cout << "============================================================\n";
    for (size_t i = 0; i < phones.size(); ++i) {
        cout << i + 1 << ". Բրենդ: " << phones[i].brand
             << " | Մոդել: " << phones[i].model
             << " | OS: " << phones[i].operatingSystem
             << " | Գին: " << fixed << setprecision(2) << phones[i].price << " դր.\n";
    }
}

// 2. Գտնել ամենաէժան հեռախոսը
```

```
void findCheapestPhone(const vector<Phone>& phones) {
    if (phones.empty()) return;
    size_t minIdx = 0;
    for (size_t i = 1; i < phones.size(); ++i) {
        if (phones[i].price < phones[minIdx].price) {
            minIdx = i;
        }
    }
    cout << "\n--- Ամենաէժան հեռախոսը ---\n";
    cout << "Բրենդ: " << phones[minIdx].brand
         << " | Մոդել: " << phones[minIdx].model
         << " | Գին: " << fixed << setprecision(2) << phones[minIdx].price << " դր.\n";
}
// 3. Արտածել տրված օպերացիոն համակարգով հեռախոսները
void printPhonesByOS(const vector<Phone>& phones, const string& targetOS) {
    cout << "\n--- Հեռախոսներ " << targetOS << " օպերացիոն համակարգով ---\n";
    bool found = false;
    for (const auto& phone : phones) {
        if (phone.operatingSystem == targetOS) {
            cout << "Բրենդ: " << phone.brand
                 << " | Մոդել: " << phone.model
                 << " | Գին: " << fixed << setprecision(2) << phone.price << " դր.\n";
            found = true;
        }
```

```
    }
    if (!found) {
        cout << "Այդպիսի օպերացիոն համակարգով հեռախոսներ չեն հայտնաբերվել:\n";
    }
}
// 4. Հաշվել տրված գնային սահմանից ցածր հեռախոսների քանակը
int countPhonesBelowPrice(const vector<Phone>& phones, double maxPrice) {
    int count = 0;
    for (const auto& phone : phones) {
        if (phone.price < maxPrice) {
            count++;
        }
    }
    return count;
}
int main() {
    setlocale(LC_ALL, "Armenian");
    // Ստեղծում ենք 10 հեռախոսից բաղկացած զանգված
    vector<Phone> phones = {
        {"Apple", "iPhone 15", "iOS", 450000.0},
        {"Samsung", "Galaxy S24", "Android", 420000.0},
        {"Xiaomi", "Redmi Note 13", "Android", 120000.0},
        {"Google", "Pixel 8", "Android", 350000.0},
        {"Apple", "iPhone 13", "iOS", 280000.0},
```

```
        {"Samsung", "Galaxy A55", "Android", 180000.0},
        {"OnePlus", "12", "Android", 310000.0},
        {"Huawei", "Pura 70", "HarmonyOS", 380000.0},
        {"Xiaomi", "Poco X6", "Android", 140000.0},
        {"Nokia", "G42", "Android", 95000.0}
    };
    // Ֆունկցիաների կանչ
    printAllPhones(phones);
    findCheapestPhone(phones);
    printPhonesByOS(phones, "Android");
    double priceLimit = 200000.0;
    int cheaperCount = countPhonesBelowPrice(phones, priceLimit);
    cout << "\n--- Գնային սահմանափակման ստուգում ---\n";
    cout << "Հեռախոսների քանակը, որոնց գինը ցածր է " << fixed << setprecision(2) <<
priceLimit << " դրամից՝ " << cheaperCount << endl;
    return 0;
}
```